

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS

COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:50

Annexure A

Debugging or Optimization task

Code of Conduct:

I N.Sai Charan(192525247)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

N.Sai Charan

Annexure A

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:

```
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)
```

Process P2:

```
lock(R2)
wait(100 ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)
```

Identify the exact condition that causes deadlock.

Suggest an optimization to eliminate the deadlock without changing process logic.

(10 Marks)

Q2. CPU Utilization Problem

A system shows low CPU utilization (30%) even though processes are ready.

You analyze the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready ...

Debug the cause using process management concepts and propose a system-level optimization to improve utilization.

(10 Marks)

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory map shows:

- Code: 5 MB

- Heap: 150 MB
- Stack: 1 MB per thread
- 25 threads running
- Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

(10 Marks)

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

- Q1 (Highest Priority): Interactive
- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

P5 is in Q3 but keeps getting postponed.

Debug the root cause of starvation and suggest an optimisation to ensure fairness.

(10 Marks)

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead.

Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

(10 Marks)

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging (25%)	Optimization (20%)	Analysis (20%)	Code Quality (15%)	Total Marks (100%)
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

Q1. Deadlock Debugging (Code Trace)

Process P1 executes `lock(R1) → wait(100ms) → lock(R2) → critical section → unlock(R2) → unlock(R1)`. Process P2 executes `lock(R2) → wait(100ms) → lock(R1) → critical section → unlock(R1) → unlock(R2)`.

1. Exact Condition Causing Deadlock

This is a classic circular-wait deadlock. All four Coffman conditions hold simultaneously:

- Mutual Exclusion – R1 and R2 are non-shareable; only one process can hold each lock at a time.
- Hold and Wait – P1 acquires and holds R1 while it waits (blocks) trying to acquire R2; symmetrically P2 holds R2 while waiting for R1.
- No Preemption – Neither the OS nor the other process can forcibly strip a lock away from a process that holds it; a lock is only released voluntarily via `unlock()`.
- Circular Wait – In the resource-allocation graph, $P1 \rightarrow (\text{waits on}) R2 \rightarrow (\text{held by}) P2 \rightarrow (\text{waits on}) R1 \rightarrow (\text{held by}) P1$, forming a closed cycle.

Timeline Trace

- $t = 0\text{ms}$: P1 executes `lock(R1)` and succeeds. P2 executes `lock(R2)` and succeeds.
- $t = 0\text{--}100\text{ms}$: Both processes execute `wait(100ms)`, simulating some processing delay before requesting the second resource.
- $t = 100\text{ms}$: P1 calls `lock(R2)` → blocks, because P2 currently holds R2. P2 calls `lock(R1)` → blocks, because P1 currently holds R1.
- $t = 100\text{ms}$ onward: Both processes are now blocked indefinitely. Neither can reach its `unlock()` calls because those calls are only reached after entering the critical section, which requires the second lock. The system is in permanent deadlock.

The 100ms wait is what makes the race almost certain: it gives both processes enough time to acquire their first lock before either attempts the second, guaranteeing the interleaving that produces the cycle.

2. Optimization to Eliminate Deadlock (Without Changing Process Logic)

Because the instruction order inside P1 and P2 cannot be touched, the fix must live outside the application code — in the lock manager / runtime layer.

Primary Fix: Global Resource Ordering (Enforced Externally)

Assign each resource a fixed global rank (e.g., $R1 = 1$, $R2 = 2$). Wrap `lock()` in a thin interception layer that reorders the actual acquisition sequence transparently: whichever resource has the lower rank is always locked first at the hardware/lock-manager level, regardless of the order the process "asked" for them in source code. Since one universal ordering is now enforced for every process, no cycle in the wait-for graph can ever form — this eliminates circular wait structurally, not just probabilistically.

Alternative / Supporting Fixes

- Try-lock with timeout and backoff: Replace blocking `lock(R2)` with a bounded-time attempt; if it fails, release R1, sleep a randomized short interval, and retry the whole sequence from the start. This converts a deadlock into a livelock-free retry loop (deadlock avoidance via preemption of held resources).

- Banker's Algorithm: If each process's maximum resource claims are declared in advance, the OS can run a safety check before granting any lock request and simply refuse to grant a request that would move the system into an unsafe state.
- Watchdog / deadlock detection: Periodically build the wait-for graph and run cycle detection; if a cycle is found, forcibly abort or roll back one of the participating processes (preemption after the fact).

Of these, global resource ordering is the preferred production answer because it is zero-overhead at run time, requires no per-request bookkeeping, and guarantees deadlock can never occur — rather than merely reducing its probability or detecting it after the fact.

3. Practical Implementation Sketch

A lock-manager wrapper could look conceptually like this (illustrative pseudocode, not part of the graded process logic):

- `acquire_ordered(R_a, R_b): if rank(R_a) < rank(R_b): lock(R_a); lock(R_b) else: lock(R_b); lock(R_a)`

Both P1's request for (R1, R2) and P2's request for (R2, R1) resolve to the same physical acquisition order — R1 then R2 — so the second process to arrive simply blocks on the first lock and waits its turn, instead of each process holding one resource and waiting on the other.

Q2. CPU Utilization Problem

Observed symptom: CPU utilization is only 30% even though the ready queue is populated, and the scheduler log shows the repeating transition pattern ready → waiting → ready → waiting → ready ...

1. Debugging the Root Cause

- I/O-bound behaviour dominating the mix: Processes are entering the waiting state almost immediately after being dispatched, which means each CPU burst is very short before an I/O request is issued. The CPU therefore spends more wall-clock time blocked or idling on interrupts than doing useful work.
- Insufficient degree of multiprogramming: If too few processes are resident in memory/ready state at once, the moment the single running process blocks on I/O there is no other ready process available to fill the gap — the CPU idles until the I/O interrupt fires and a process becomes ready again.
- Poor overlap of CPU and I/O activity: The log pattern suggests processes are not issuing I/O requests asynchronously or in bulk; each I/O call fully blocks the calling process, serializing CPU and I/O phases instead of overlapping them across processes.
- Possible quantum mismatch: If a round-robin quantum is set without regard to typical burst length, the scheduler may be needlessly preempting or mis-scheduling around very short bursts, amplifying the waiting time relative to running time.

2. System-Level Optimization

- Increase the degree of multiprogramming: Load more processes into main memory (raise the multiprogramming level) so that whenever one process blocks for I/O, the scheduler always has another ready process to dispatch immediately, keeping the CPU busy.

- Adopt a Multilevel Feedback Queue (MLFQ): Let processes that repeatedly return to ready quickly (I/O-bound) get boosted to a higher-priority queue so they are dispatched instantly on becoming ready, minimizing the time between their I/O completion and their next brief CPU burst — improving overlap system-wide.
- Enable asynchronous / non-blocking I/O: Allow a process to issue several I/O requests without blocking on each one individually, reducing the frequency of ready↔waiting transitions and letting the CPU scheduler batch useful work between I/O completions.
- Retune the time quantum: Set the quantum based on measured average burst time so that genuinely CPU-bound work is not thrashed by oversized context-switch frequency, while I/O-bound work is still context-switched out promptly when it blocks.

Expected result: with more resident processes and quicker re-dispatch of I/O-bound work, the CPU has a ready process available almost any time the current one blocks, which should push utilization well above the observed 30% toward the 70–90% range typical of a well-tuned multiprogrammed system.

Q3. Memory Bottleneck Debugging

1. Memory Footprint Calculation

Segment	Size
Code	5 MB
Heap	150 MB
Stack (25 threads × 1 MB)	25 MB
Total per-process footprint	180 MB

2. Identifying the Memory Issue

180 MB is small compared to the available 2 GB of physical RAM, so raw size alone is not the bottleneck — the problem lies in memory access behaviour and competition for physical frames:

- Thrashing from working-set pressure: If several instances of this program run concurrently, or the system has other memory-resident processes, the combined demand can exceed available physical frames, forcing the OS to continuously swap heap pages in and out — the classic definition of thrashing, where more time is spent paging than executing.
- Poor locality of reference in the 150 MB heap: With 25 threads scattered across a large shared heap, each thread's working set (the pages it actively touches) may span far more of the 150 MB region than necessary, meaning many distinct pages must stay resident simultaneously — if physical frames allocated to the process are fewer than this working set, page faults spike.
- Heap fragmentation / false sharing: Threads allocating and freeing objects concurrently in a shared heap can fragment memory, inflating the effective number of pages touched even though logical memory usage stays the same, and can also cause cache-line ‘false sharing’ that degrades performance further (though this is a CPU-cache issue rather than a paging issue, it compounds the slowdown).

3. Two Optimizations

- Improve locality with pooled/arena allocation: Give each thread (or small groups of threads) its own memory arena/pool instead of allocating from one shared 150 MB heap. This keeps each thread's active pages compact and localized, shrinking the effective working set and reducing the number of page faults per unit of work.
- Right-size concurrency and paging policy: Replace 25 raw threads with a bounded thread pool sized to actual available parallelism (e.g., matching core count, such as 8), which cuts total concurrent memory pressure; in parallel, ensure the page-replacement algorithm (e.g., an LRU-approximation like the clock algorithm) and per-process frame allocation are tuned so each process's working set fits comfortably in its allotted physical frames.

Secondary measures worth mentioning: reducing per-thread stack size where safe, and using memory-mapped I/O or lazy allocation for the heap so only touched pages are actually backed by physical memory.

Q4. Starvation Debugging

Setup: a strict multilevel queue scheduler with Q1 (Interactive, quantum 5ms) as highest priority, Q2 (System, quantum 10ms) next, and Q3 (Batch, FCFS) lowest. P5 resides in Q3 and is repeatedly postponed.

1. Root Cause of Starvation

- Strict, non-preemptible priority across queues: A multilevel queue (as opposed to a multilevel feedback queue) assigns each process to a fixed queue permanently — there is no mechanism for a process to move between queues based on wait time or behaviour.
- Absolute precedence: The scheduler only services Q3 when both Q1 and Q2 are completely empty. Under any sustained load of interactive or system processes, Q1 and Q2 rarely (if ever) drain simultaneously, so Q3 — and therefore P5 — may never be scheduled at all.
- FCFS within Q3 compounds the problem: Even in the rare moments Q3 does get CPU time, P5 must also wait behind every process ahead of it in FCFS order before it is chosen, adding a second layer of delay on top of the queue-level starvation.

This is textbook indefinite/priority starvation: P5 is fully ready and technically schedulable, yet the scheduling policy provides no path for it to ever accumulate priority or force its way onto the CPU.

2. Optimization to Ensure Fairness

- Convert to a Multilevel Feedback Queue (MLFQ) with aging: Allow a process's effective priority to increase the longer it waits without being scheduled. After a defined wait threshold, P5 would be promoted from Q3 into Q2 (or even Q1), guaranteeing it eventually receives CPU time regardless of ongoing load in the higher queues.
- Guaranteed minimum CPU share per queue (weighted round-robin across queues): Instead of strict precedence, allocate a fixed percentage of CPU time slices to each queue every scheduling cycle (e.g., Q1 = 60%, Q2 = 30%, Q3 = 10%) so Q3 — and thus P5 — always makes forward progress even under continuous higher-queue load.

Either fix (aging-based promotion, or a guaranteed minimum quota) removes the possibility of indefinite postponement while still respecting the general intent of prioritizing interactive and system work.

Q5. Context-Switch Overhead Debugging

Observed: 20,000 context switches/sec, average burst time of 2ms per thread, very high I/O wait frequency, and a

preemptive scheduler with a 1ms quantum.

1. Why Context Switching Is So High

- Quantum too close to burst time: With a 1ms quantum against a 2ms average burst, most threads are preempted mid-burst — roughly halfway through their useful work — forcing an extra forced switch for nearly every thread that would otherwise have finished on its own.
- High I/O wait frequency adds a second source of switching: Threads that block on I/O before even using their full quantum trigger an additional voluntary context switch on top of the preemptive ones, effectively doubling the switching sources acting on the system simultaneously.
- Overhead-to-work ratio inverts: Each context switch consumes fixed overhead (register/state save-restore, TLB flush, cache pollution, scheduler bookkeeping). When the quantum (1ms) is on the same order as this overhead plus the burst time, the fraction of CPU time spent switching rather than computing becomes very large — explaining why 20,000 switches/sec produces high overhead relative to the actual 2ms bursts being completed.

2. Two Optimizations to Reduce Overhead

- Increase the time quantum: Raise the quantum to comfortably exceed the average burst time (e.g., 4–5ms), so the large majority of bursts complete naturally within a single quantum instead of being preemptively cut off, directly cutting the preemption-driven share of the 20,000 switches/sec.
- Adopt a scheduling policy tuned for short, I/O-bound bursts: Use a Multilevel Feedback Queue that keeps short-burst, I/O-heavy threads in a queue with quick re-dispatch on wake-up but does not force a full preemptive quantum cycle for work that is already naturally yielding the CPU via I/O calls; alternatively, batch or group similar short-burst threads and use asynchronous I/O so threads issue requests without repeatedly blocking and triggering separate wake/switch cycles.

Combined effect: raising the quantum removes most premature preemptions, while an I/O-aware scheduling policy reduces the number of voluntary switches triggered by blocking calls — together bringing the switch rate down from a scheduling-overhead-dominated regime toward one where the CPU spends the bulk of its cycles on actual thread execution.

